



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

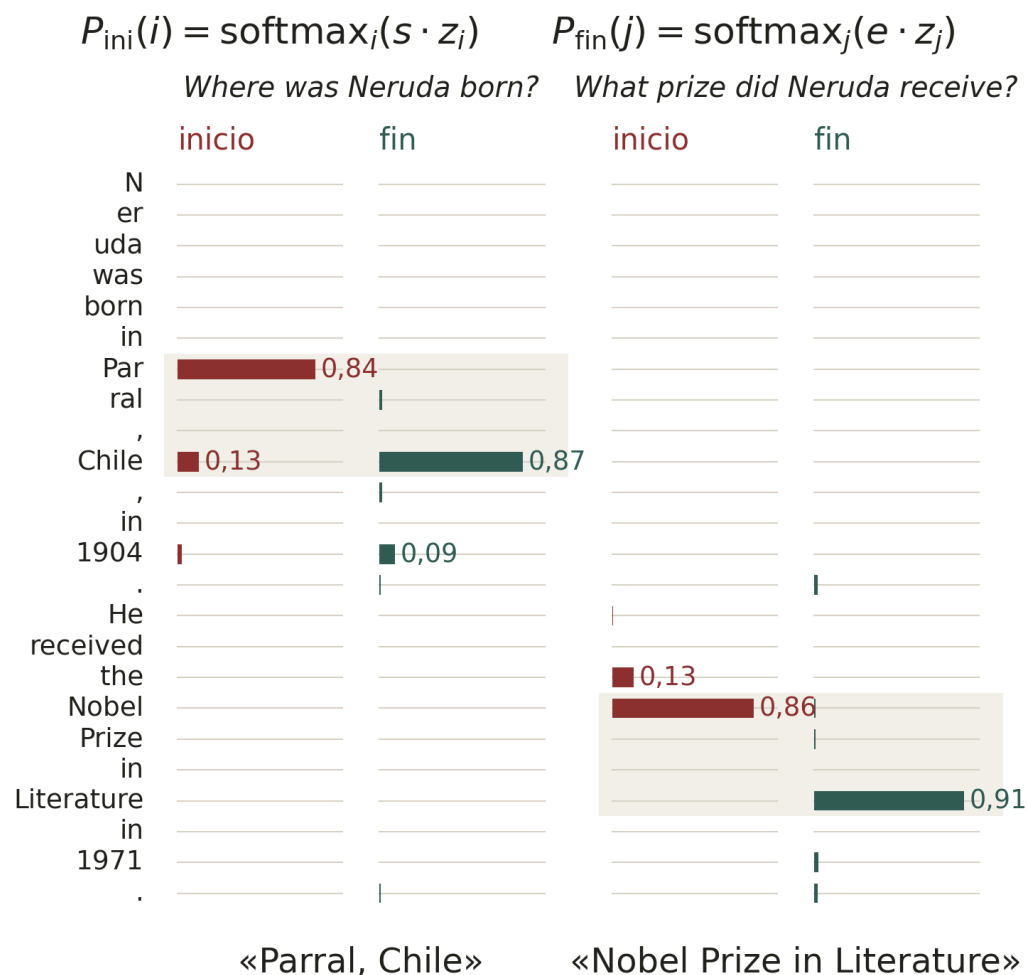
Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

Extracción de respuesta

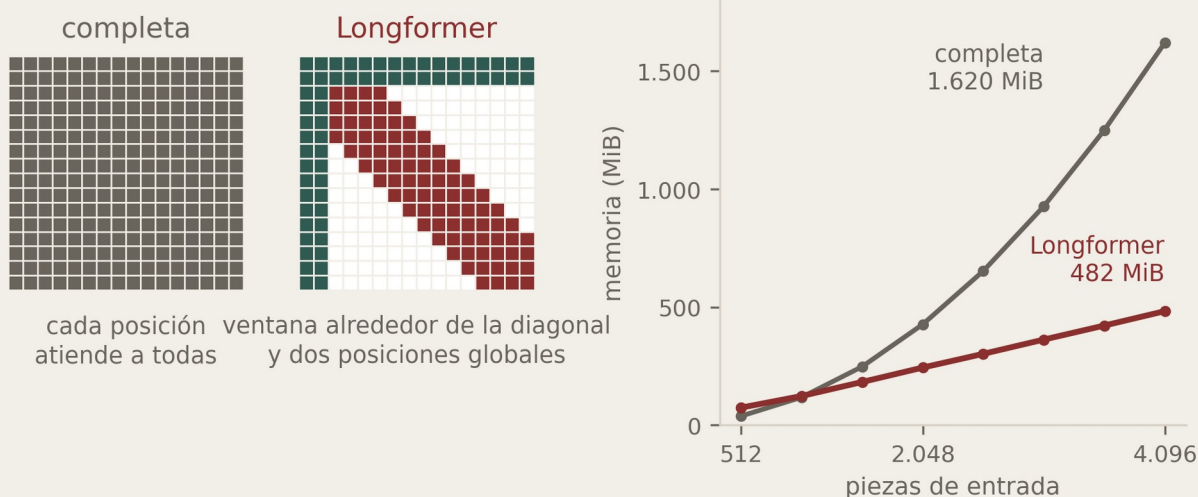
Los vectores s y e son los mismos para las dos preguntas. Lo que cambia son las salidas z_i , porque la pregunta y el párrafo pasan juntos por el encoder.



Longformer

RoBERTa con otra atención. Cada posición atiende a una ventana a su alrededor y unas pocas posiciones atienden a todas, así que la memoria crece de forma lineal con el largo de la entrada.

	RoBERTa	Longformer
atención	completa, $n \times n$	ventana de 512 más posiciones globales
ventana de entrada	512 piezas	4.096 piezas
posiciones	512, aprendidas	las mismas 512, copiadas ocho veces
proyecciones	Q, K, V	Q, K, V locales y otras tres globales
preentrenamiento	desde cero	continúa desde RoBERTa
parámetros	124,6 M	148,7 M, con 2,8 M de posiciones y 21,3 M globales



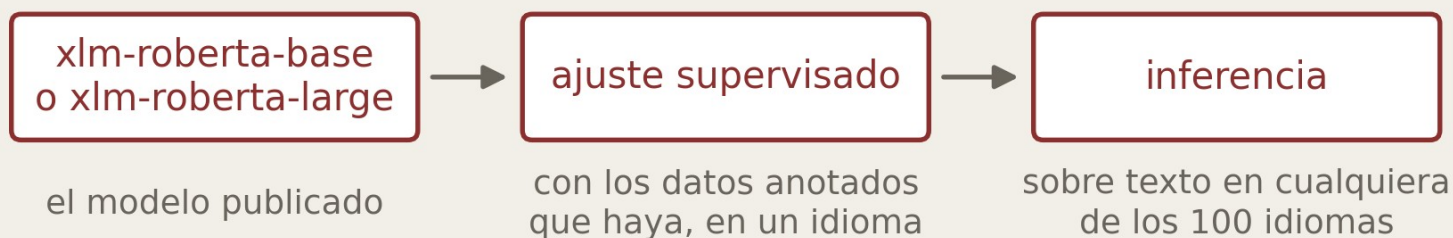
Beltagy, Peters y Cohan, 2020. Memoria de activaciones medida en inferencia sobre una secuencia, contra la misma red con atención completa.

XLm-RoBERTa

La receta de RoBERTa aplicada a 100 idiomas, con un corpus de CommonCrawl en lugar de Wikipedia. Se usa igual que mBERT.

idiomas	100
corpus	CC-100: 2,5 TB de CommonCrawl filtrado, no Wikipedia
vocabulario	SentencePiece compartido, 250.002 piezas
muestreo	suavizado exponencial con $\alpha = 0,3$; mBERT usaba 0,7
arquitectura	la de RoBERTa, solo masked LM; las doce capas pesan 85.054.464
tabla de embeddings	69,1 % del modelo base; en mBERT, 51,6 %

cómo se usa



03

Decoders de texto

Cómo se entrena un modelo de lenguaje y cómo se genera texto con él

Modelos de lenguaje

Un modelo de lenguaje asigna una probabilidad a cada palabra del vocabulario como continuación de un contexto.

$$e = [Ex_{t-3}; Ex_{t-2}; Ex_{t-1}]$$

la misma E para las tres palabras

$$h = g(We + b)$$

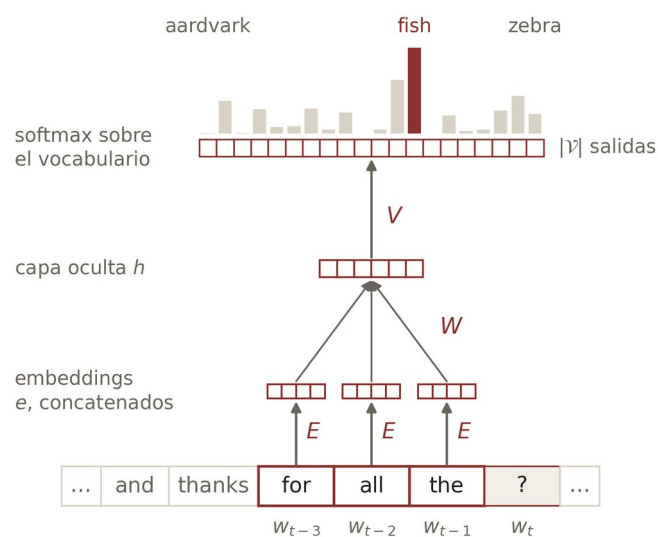
capa oculta

$$\hat{y} = \text{softmax}(Vh)$$

una probabilidad por palabra del vocabulario

$$\hat{y}[w] = P(w_t = w \mid w_{t-3}, w_{t-2}, w_{t-1})$$

eso es un modelo de lenguaje



El contexto es una ventana

Las tres palabras anteriores, cada una con su embedding.

La salida es una distribución

Una softmax sobre todo el vocabulario.

E se comparte

La misma matriz de embeddings para las tres posiciones.

Next token prediction

El modelo se entrena prediciendo la palabra siguiente de un texto. Las etiquetas salen del propio texto, así que el entrenamiento es autosupervisado.

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \sum_{w \in \mathcal{V}} y_t[w] \log \hat{y}_t[w] = -\frac{1}{T} \sum_{t=1}^T \log \hat{y}_t[\mathbf{w}_{t+1}]$$

y_t es one-hot en la palabra siguiente del texto, $\mathbf{w}_{t+1}$

so	long	and	thanks	for	all	the	fish	$-\log P(\text{thanks} \mid \text{so long and})$
so	long	and	thanks	for	all	the	fish	$-\log P(\text{for} \mid \text{long and thanks})$
so	long	and	thanks	for	all	the	fish	$-\log P(\text{all} \mid \text{and thanks for})$
so	long	and	thanks	for	all	the	fish	$-\log P(\text{the} \mid \text{thanks for all})$
so	long	and	thanks	for	all	the	fish	$-\log P(\text{fish} \mid \text{for all the})$

entrada etiqueta: la palabra siguiente del texto

+ un término por posición del corpus

Cada posición es un ejemplo

El contexto es la entrada y la palabra que sigue es la etiqueta.

La pérdida es la entropía cruzada

Con una etiqueta one-hot queda un solo término por posición.

Sirve cualquier corpus

No hace falta anotar nada.

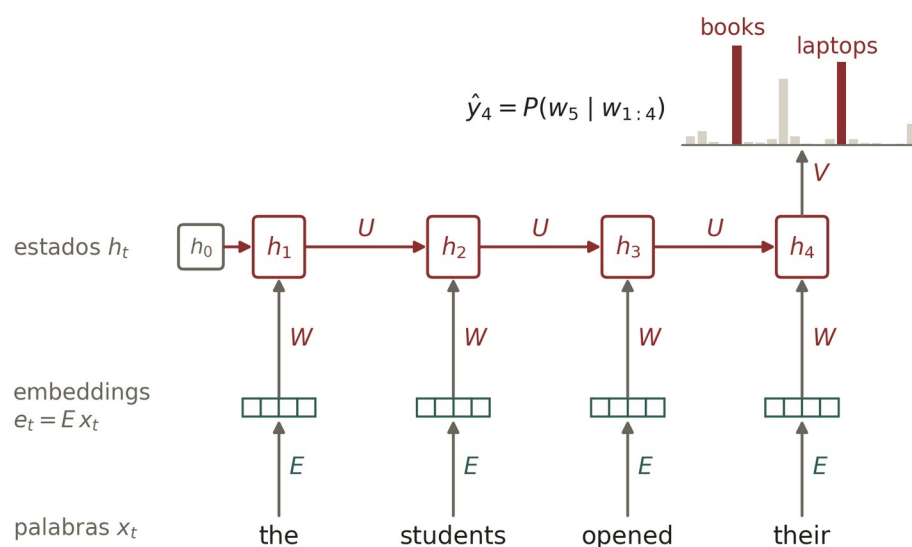
Modelo de lenguaje recurrente

La red recurrente de la clase 4 con una softmax sobre el vocabulario. El estado h_t resume todo el prefijo, así que la ventana deja de ser fija.

$$e_t = E x_t \quad \text{embedding de la palabra } t$$

$$h_t = g(W e_t + U h_{t-1}) \quad \text{lo de ahora y lo que traía}$$

$$\hat{y}_t = \text{softmax}(V h_t) \quad \hat{y}_t[w] = P(w_{t+1} = w | w_{1:t})$$



Sin ventana fija

h_t acumula la información de las palabras 1 a t .

Las mismas matrices

W , U y V se comparten entre posiciones.

Una salida por posición

$\hat{y}_t$ es la distribución de la palabra $t + 1$.

Next token prediction sobre el transformer

Un transformer con atención causal implementa el modelo de lenguaje. Se entrena con next token prediction y se llama causal language model.

$$\text{mask}_{ij} = -\infty \text{ si } j > i$$

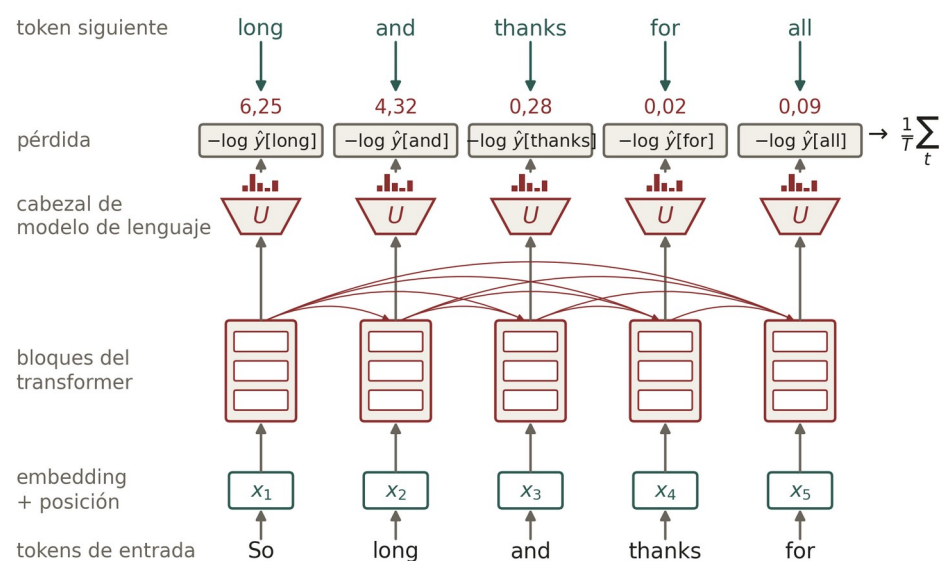
atención causal

$$\hat{y}_t = \text{softmax}(\mathbf{U} h_t)$$

h_t : salida del último bloque en la posición t

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log \hat{y}_t[w_{t+1}]$$

las T posiciones en una sola pasada



Radford et al., 2018; Jurafsky y Martin, cap. 7. Pérdidas medidas con Mistral-7B.

Ecuaciones de transferencia del decoder

Se apilan L bloques y la salida del último pasa por el cabezal U.

$x_i = e_i + \text{PE}(i)$	embedding más posición
$a_i = \text{MultiHead}(x_i, [x_1, \dots, x_j]), \quad j \leq i$	atención causal
$z_i = \text{LayerNorm}(x_i + a_i)$	residual y normalizar
$f_i = W_2 \max(0, W_1 z_i)$	red por posición
$y_i = \text{LayerNorm}(z_i + f_i)$	salida del bloque, entrada del siguiente
$h_i = y_i^{(L)}$	salida del último de los L bloques
$\hat{y}_i = \text{softmax}(\mathbf{U} h_i)$	cabezal de modelo de lenguaje
$\mathcal{L} = -\frac{1}{T} \sum_{i=1}^T \log \hat{y}_i[w_{i+1}]$	next token prediction

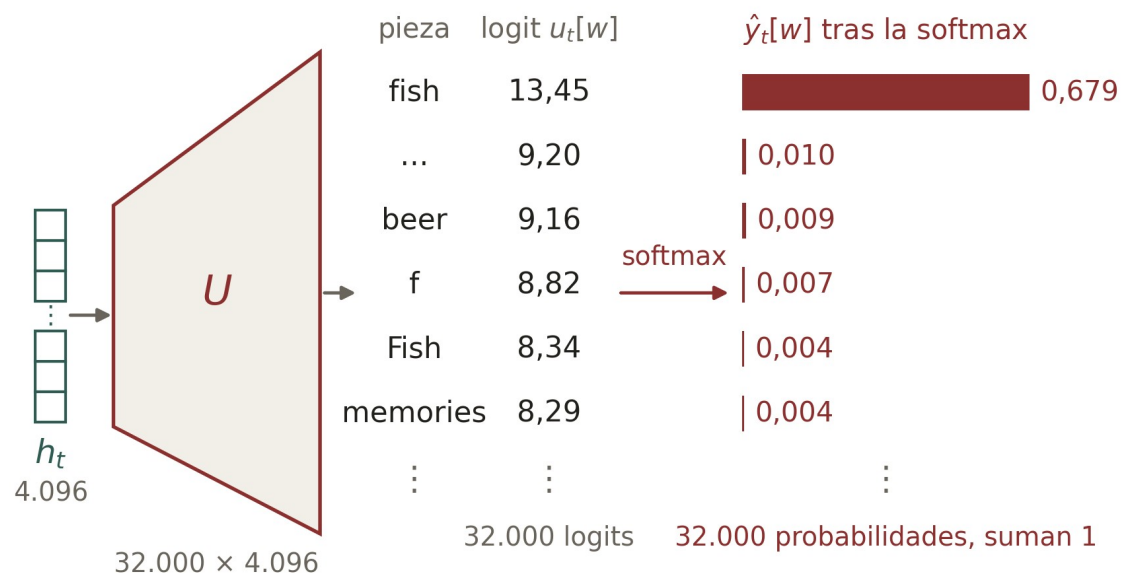
El cabezal de modelo de lenguaje

U lleva h_t de la dimensión del modelo al tamaño del vocabulario. La softmax convierte esos logits en probabilidades, que forman una distribución sobre el vocabulario.

$$u_t = U h_t, \quad U \in \mathbb{R}^{|\mathcal{V}| \times d}, \quad h_t \in \mathbb{R}^d, \quad u_t \in \mathbb{R}^{|\mathcal{V}|} \quad \text{un logit por pieza del vocabulario}$$

$$\hat{y}_t[w] = \frac{e^{u_t[w]}}{\sum_{v \in \mathcal{V}} e^{u_t[v]}} \quad \text{positivos y suman 1}$$

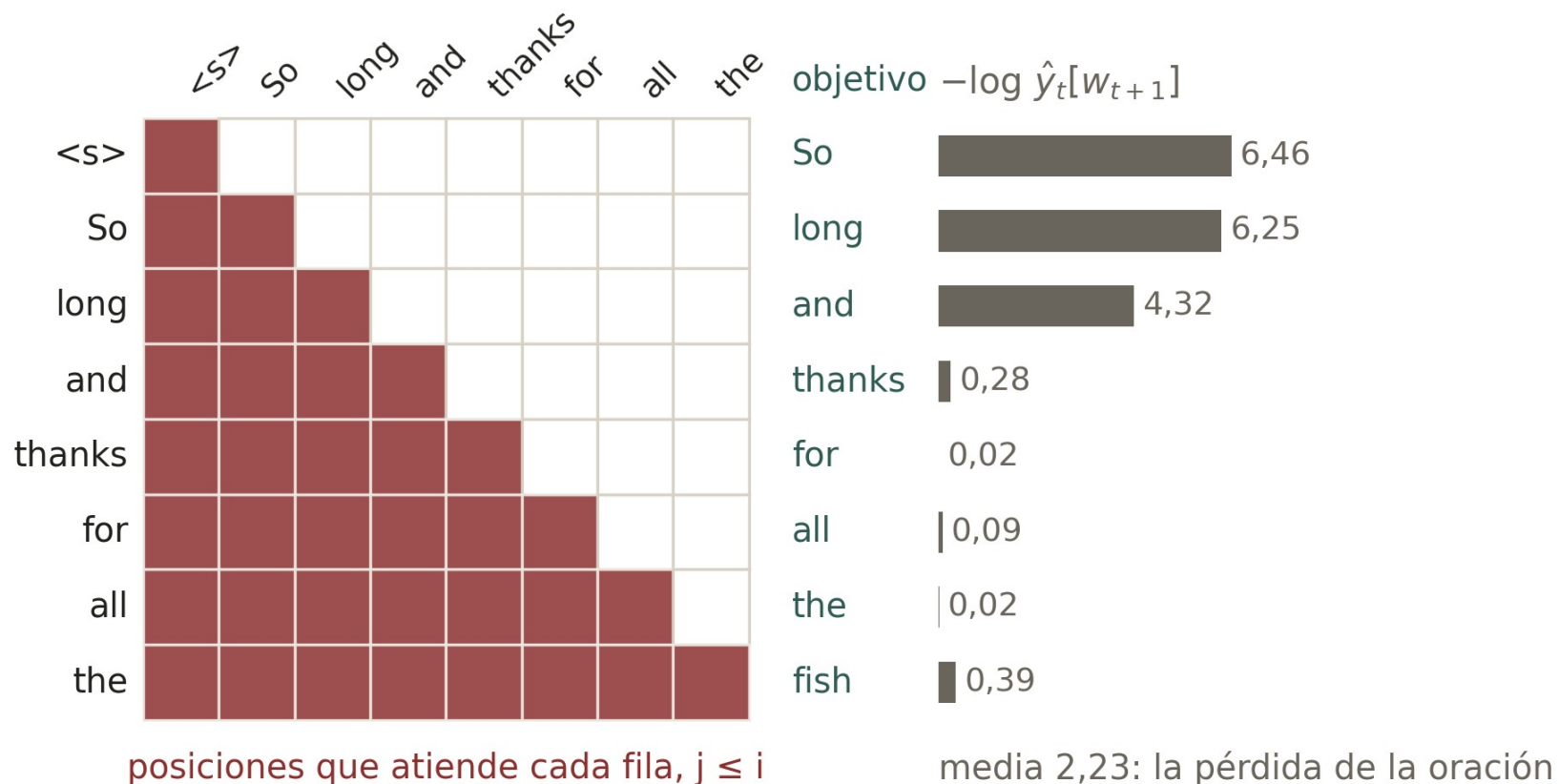
$$\hat{y}_t[w] = P(w_{t+1} = w \mid w_{\leq t}) \quad \text{una distribución sobre el vocabulario}$$



Mistral-7B, última posición de «So long and thanks for all the». El vocabulario es de piezas de BPE, así que en rigor la distribución es sobre piezas y no sobre palabras.

Entrenamiento en paralelo

La máscara causal impide que cada posición vea su objetivo. Por eso las T posiciones de una secuencia se entrenan en una sola pasada, y cada fila es un ejemplo distinto.



Pérdidas medidas con Mistral-7B sobre la oración completa, en una pasada.

Generación autorregresiva

Un modelo de lenguaje preentrenado genera texto si el token que produce en una posición se usa como entrada de la posición siguiente.

$$x_{t+1} = w_{t+1}$$

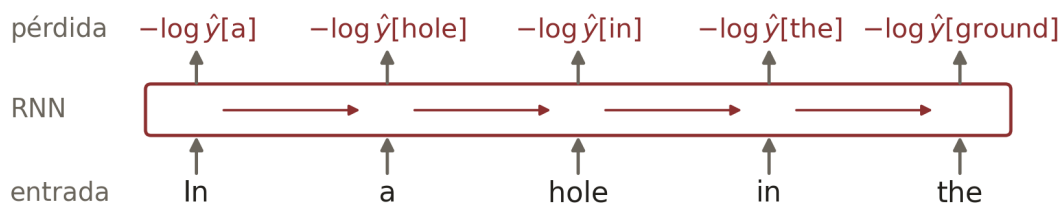
entrenamiento: la palabra del texto

$$\hat{w}_{t+1} \sim \hat{y}_t,$$

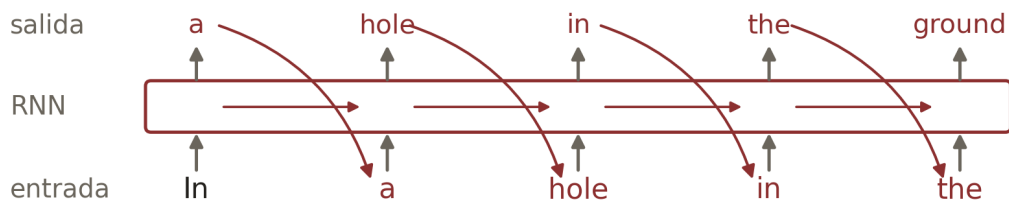
$$x_{t+1} = \hat{w}_{t+1}$$

inferencia: la palabra generada

entrenamiento: la entrada es el texto



inferencia: la entrada es lo generado



En el entrenamiento

La entrada de cada posición es la palabra del texto.

En la inferencia

La entrada es la palabra que el modelo acaba de generar.

Se repite hasta EOS

O hasta un largo máximo.

GPT-1

Un decoder preentrenado con next token prediction y ajustado después para cada tarea, como BERT. L1 se usa en las dos etapas: en el ajuste se suma a la pérdida de la tarea.

$$L_1(\mathcal{U}) = - \sum_t \log P(w_t \mid w_{t-k}, \dots, w_{t-1})$$

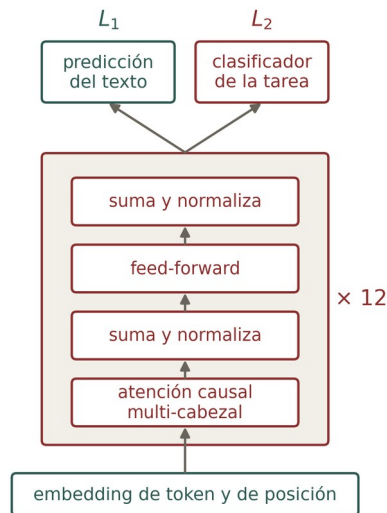
preentrenamiento: NTP sobre el corpus $\mathcal{U}$

$$L_2(\mathcal{C}) = - \sum_{(x,y) \in \mathcal{C}} \log \text{softmax}(h_m W_y)[y]$$

ajuste: la tarea sobre los datos anotados $\mathcal{C}$

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda L_1(\mathcal{C})$$

en el ajuste, NTP sigue: sobre los textos de $\mathcal{C}$, $\lambda=0,5$



Preentrenamiento

Solo L1, next token prediction sobre BooksCorpus.

Ajuste

Se minimiza L3: la tarea, L2, más λ por L1 calculada sobre los textos de la tarea.

Sin NSP

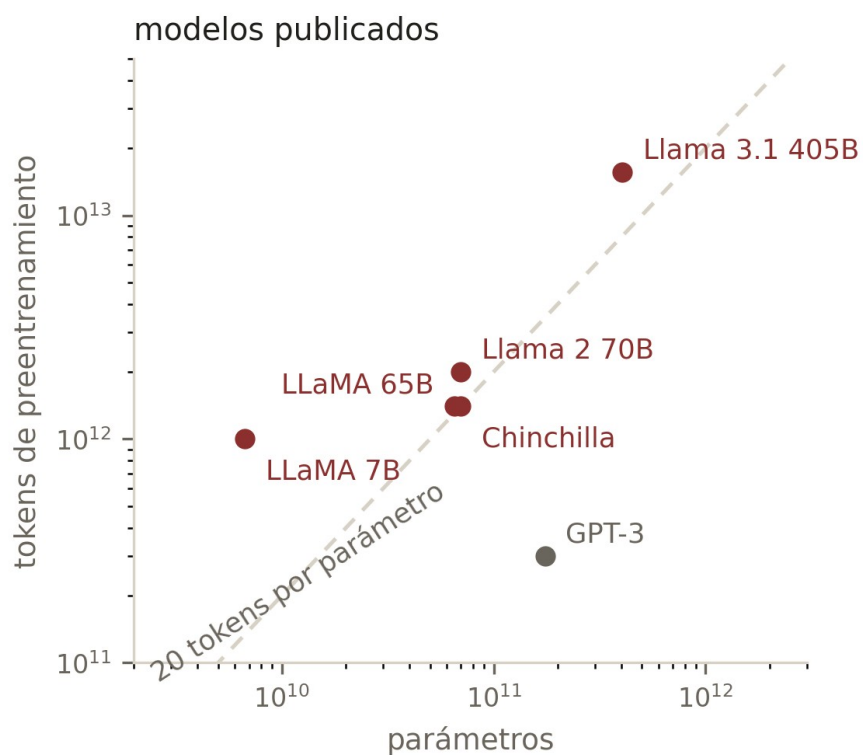
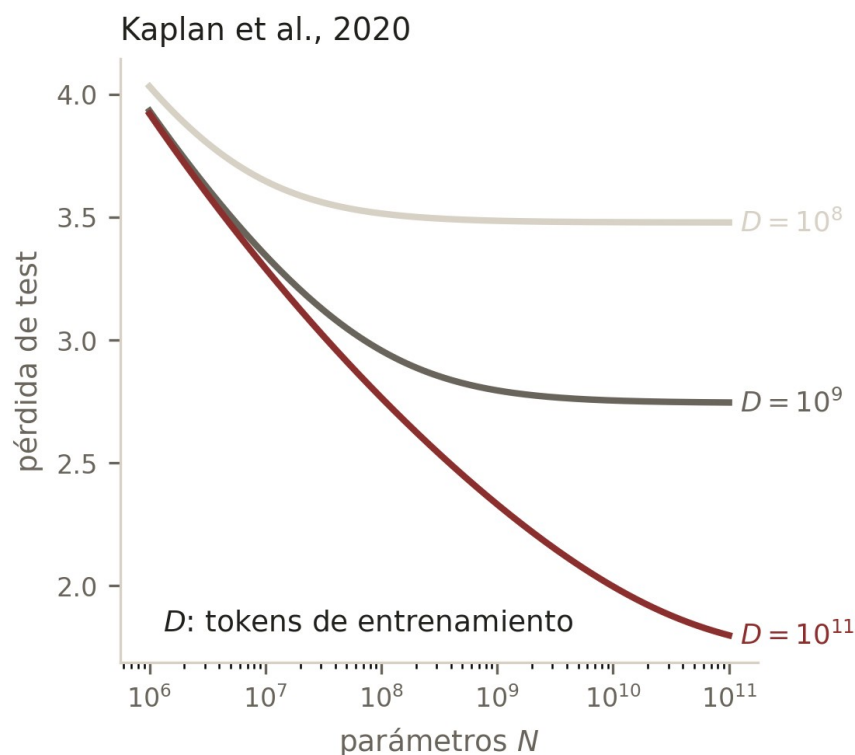
La segunda tarea es la supervisada, no una de preentrenamiento.

Tamaño

12 capas, 116,5 M de parámetros, ventana de 512.

Large Language Models

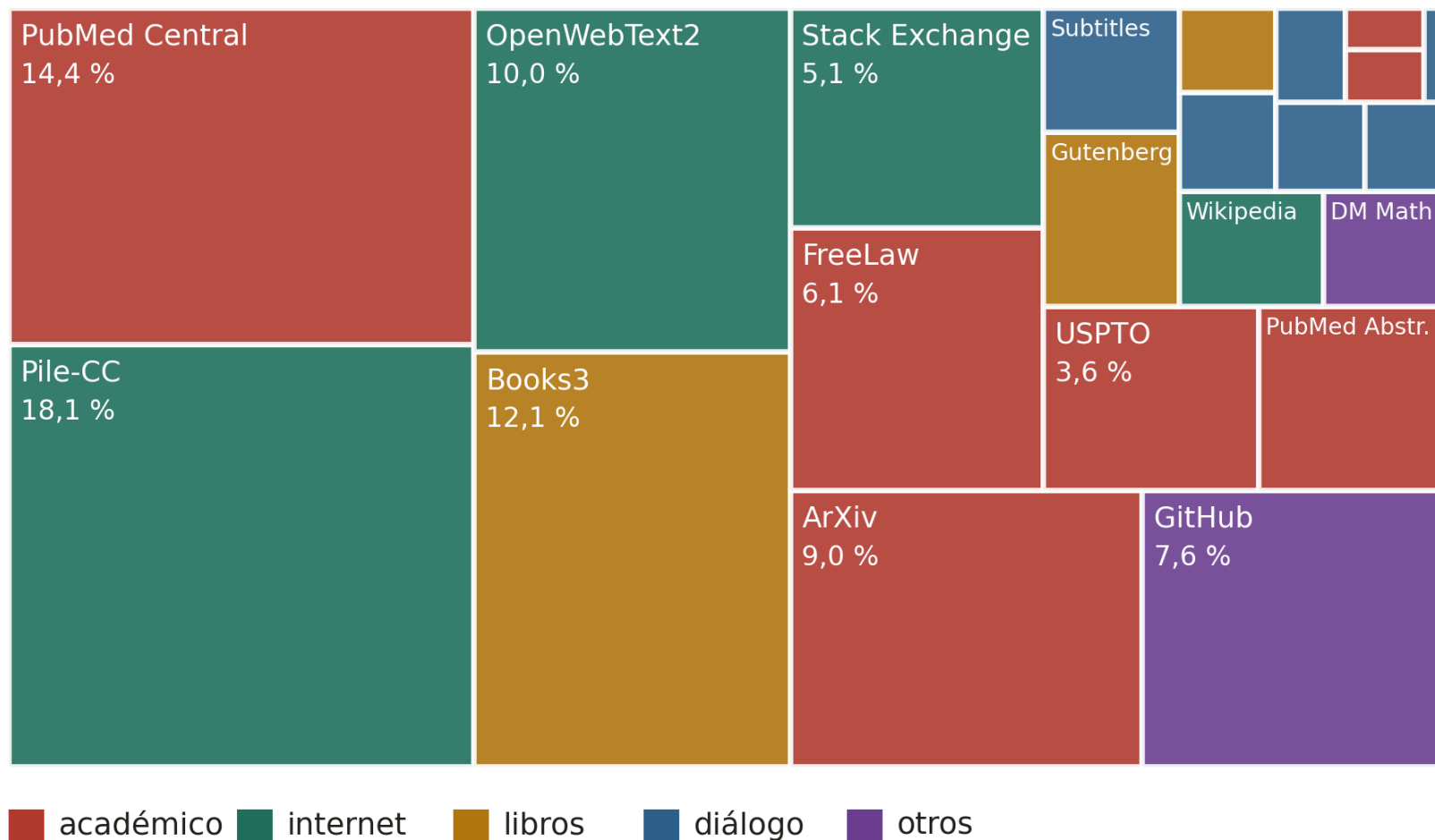
La pérdida baja si crecen a la vez los parámetros N y los tokens de entrenamiento D . Los modelos preentrenados así sobre volúmenes enormes de texto se llaman LLM.



Kaplan et al., 2020, ec. 1.5; Brown et al., 2020; Hoffmann et al., 2022; Touvron et al., 2023a y 2023b; Llama Team, 2024.

Datos masivos

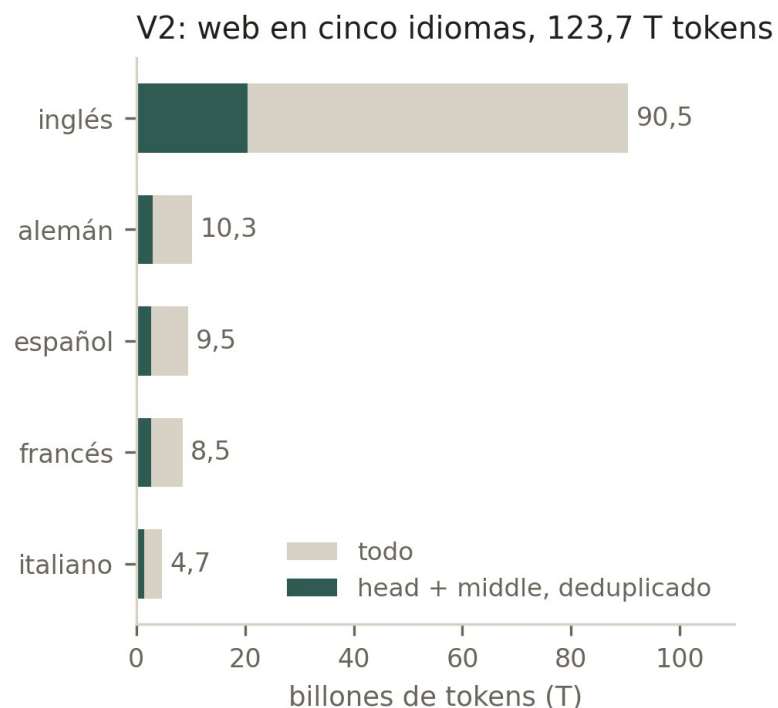
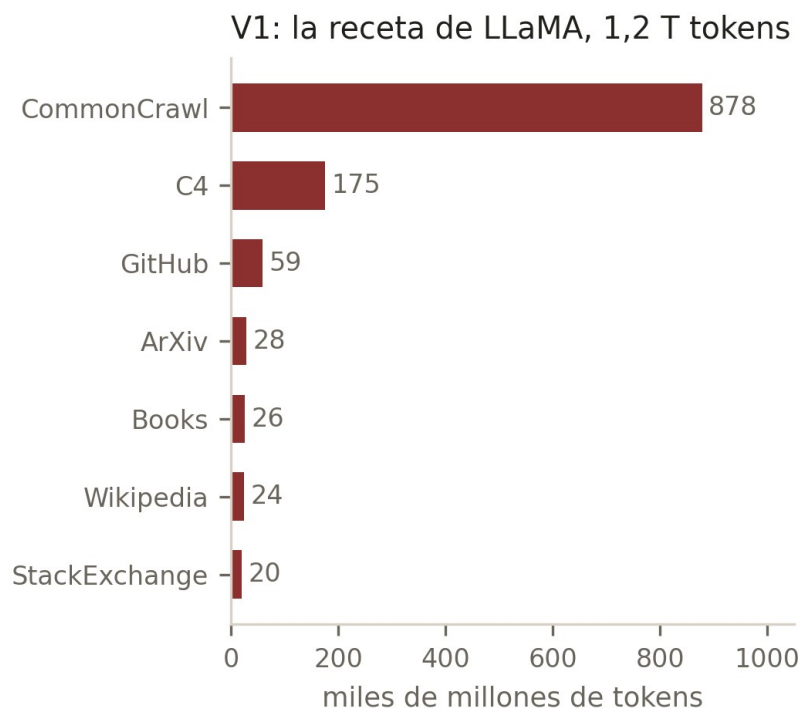
The Pile reúne 22 corpus en inglés, 825 GiB, con variedad temática. El área de cada rectángulo es su peso en el preentrenamiento.



Gao et al., 2020. Treemap recalculado desde los pesos publicados.

RedPajama

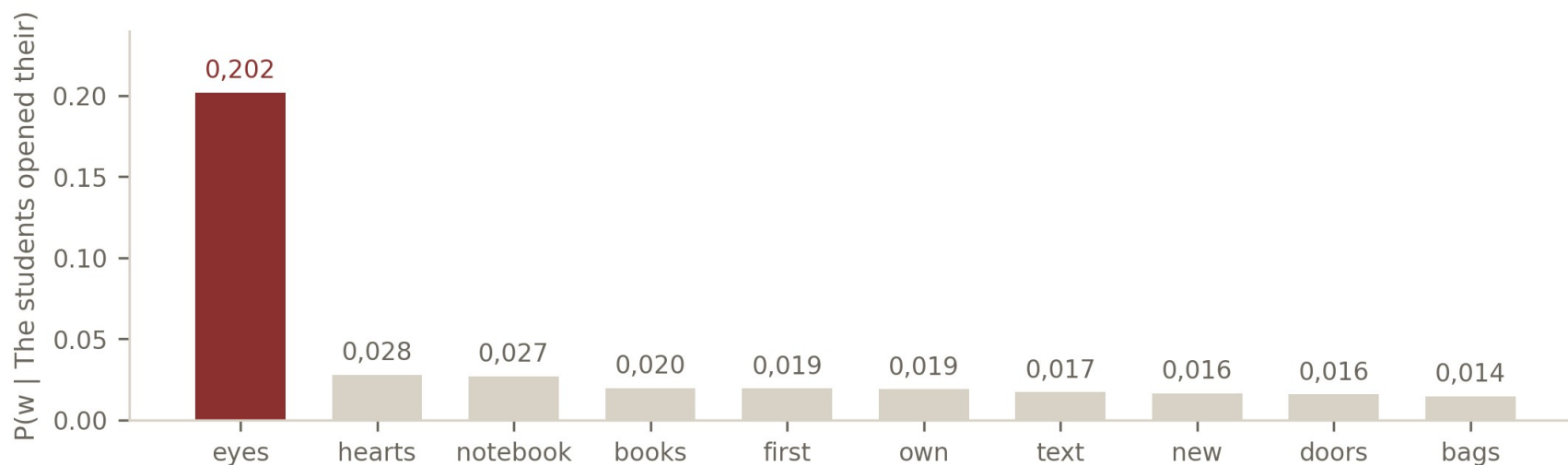
V1 reproduce los datos de LLaMA con fuentes abiertas. V2 es web en cinco idiomas, sin filtrar, con señales de calidad por documento.



Greedy decoding

En cada paso se elige el token más probable. Es determinista: con el mismo contexto genera siempre el mismo texto.

$$\hat{w}_t = \operatorname{argmax}_{w \in \mathcal{V}} P(w \mid \mathbf{w}_{<t}) \quad \text{en cada paso, la pieza más probable dado lo ya generado}$$

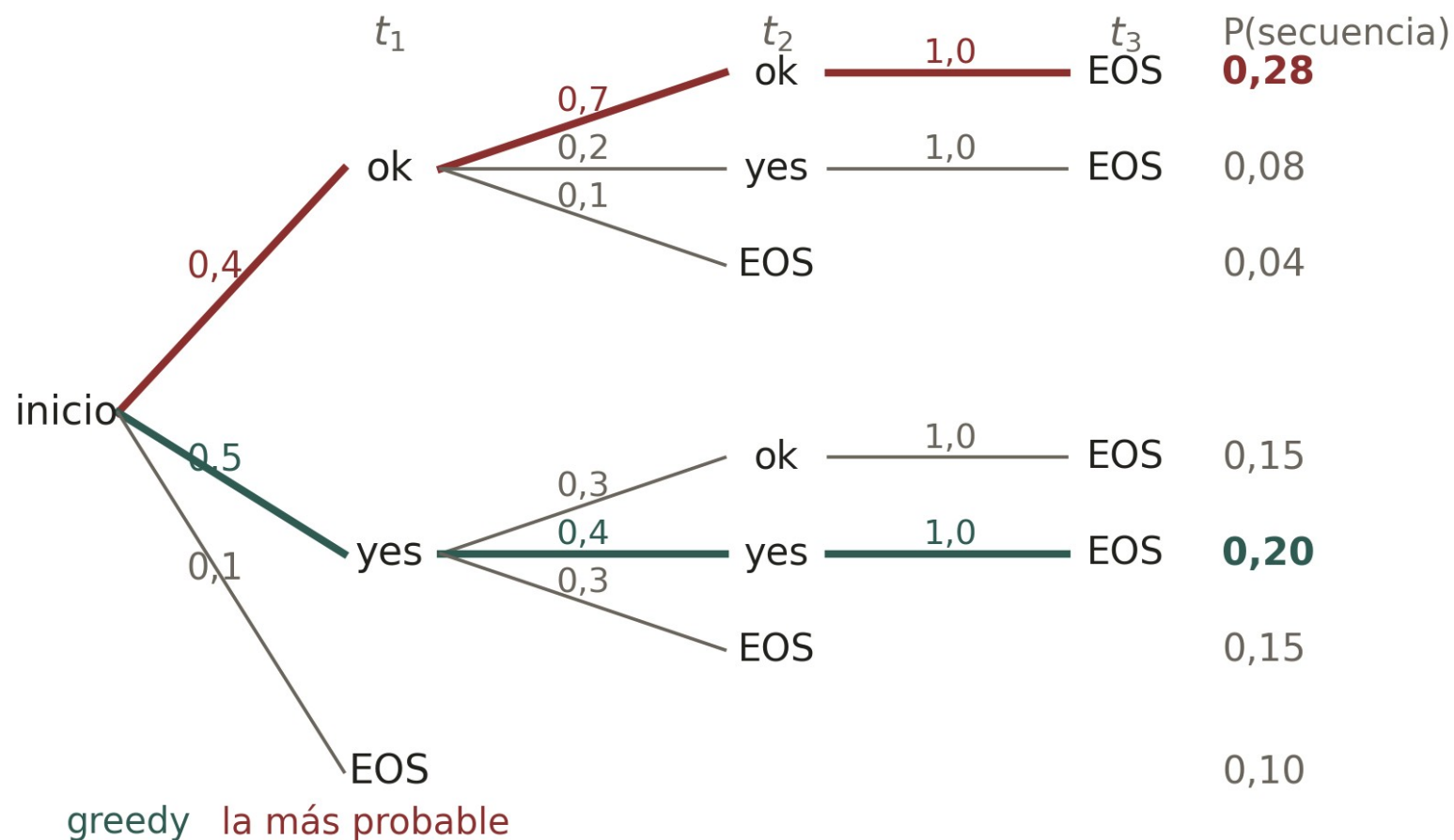


greedy: *The students opened their eyes and looked around. They were in a large room with a high ceiling. The walls were made of stone and the floor was covered with a thick layer of dust. There were no ...*

Distribución y continuación medidas con Mistral-7B.

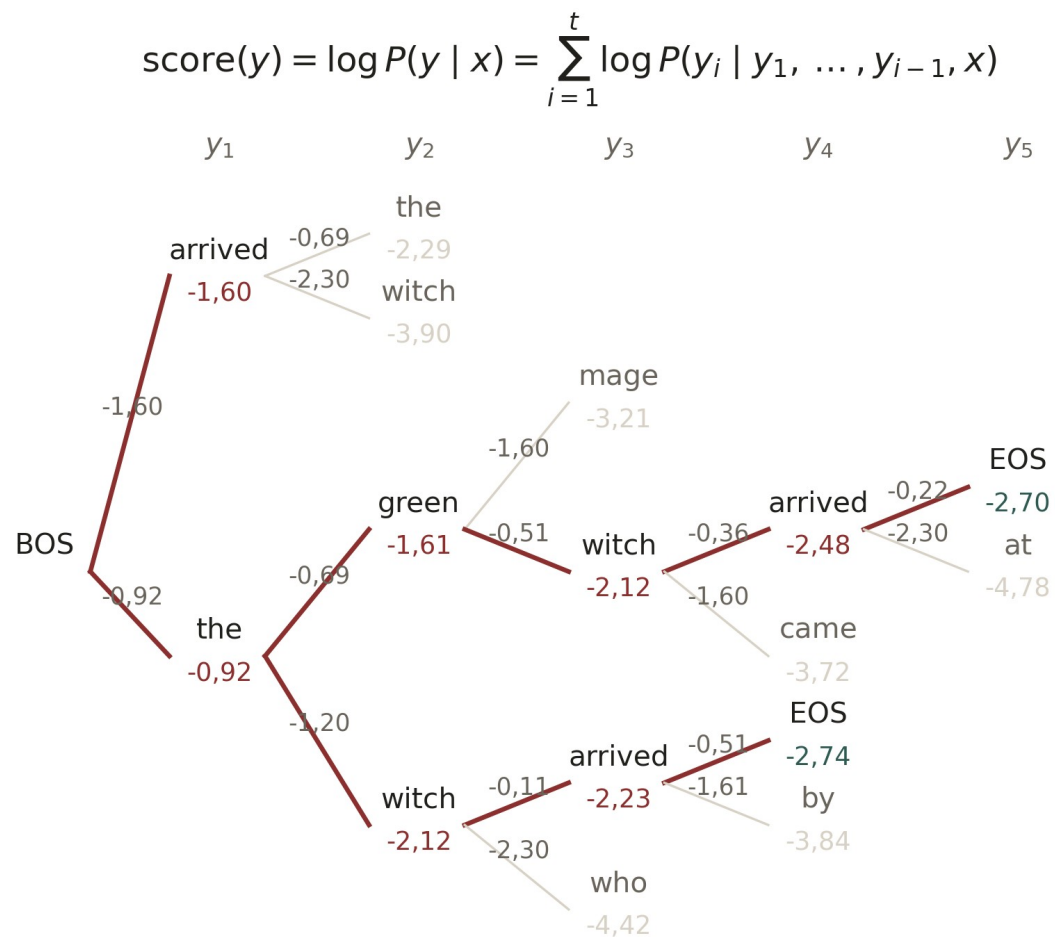
La secuencia más probable

Greedy elige yes porque en el primer paso es lo más probable. La secuencia más probable empieza por ok.



Beam search

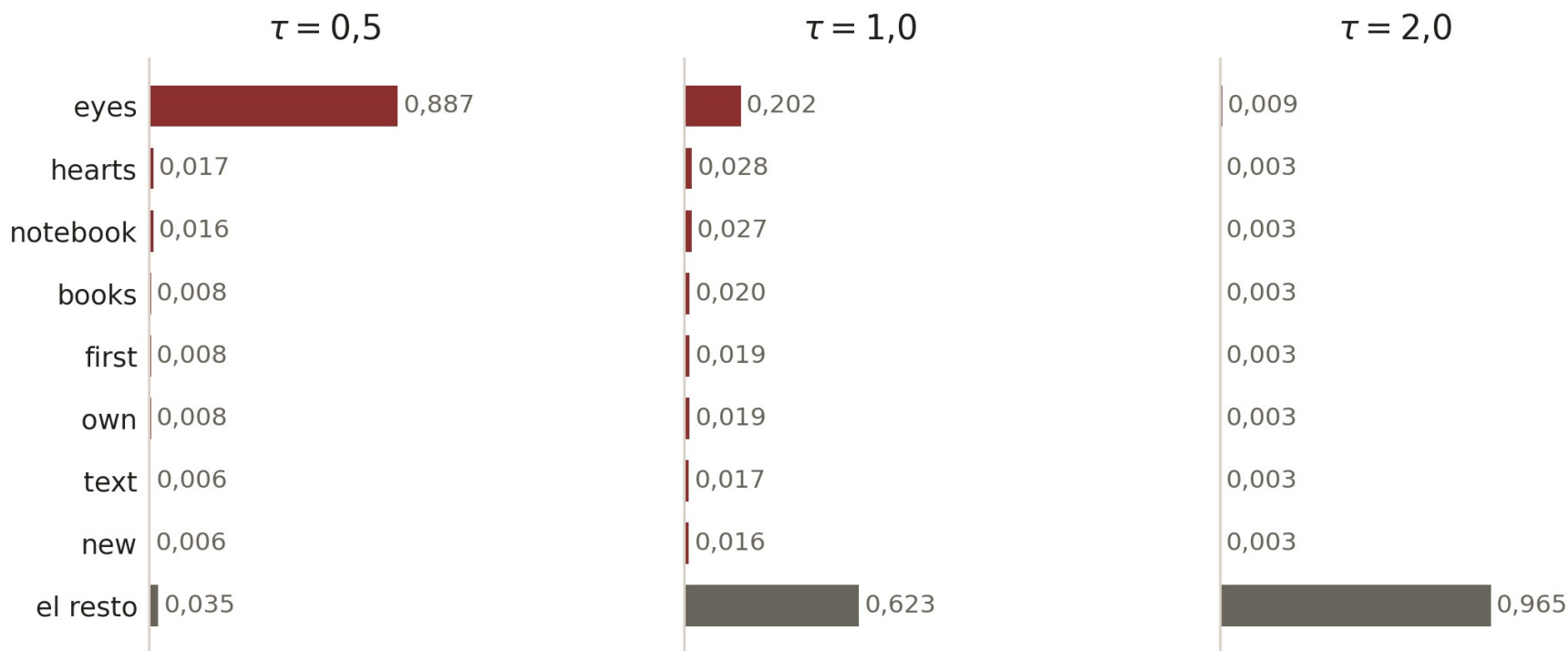
En cada paso se expanden las k hipótesis de mejor puntaje y se descartan las demás. Aquí k = 2; en gris, las hipótesis podadas.



Temperatura

En vez de elegir el token más probable, se muestrea de la distribución. La temperatura divide los logits antes de la softmax y controla cuán concentrada queda.

$$\hat{y} = \text{softmax}(u/\tau) \quad u: \text{los logits}; \tau < 1 \text{ concentra}, \tau > 1 \text{ aplanada}$$



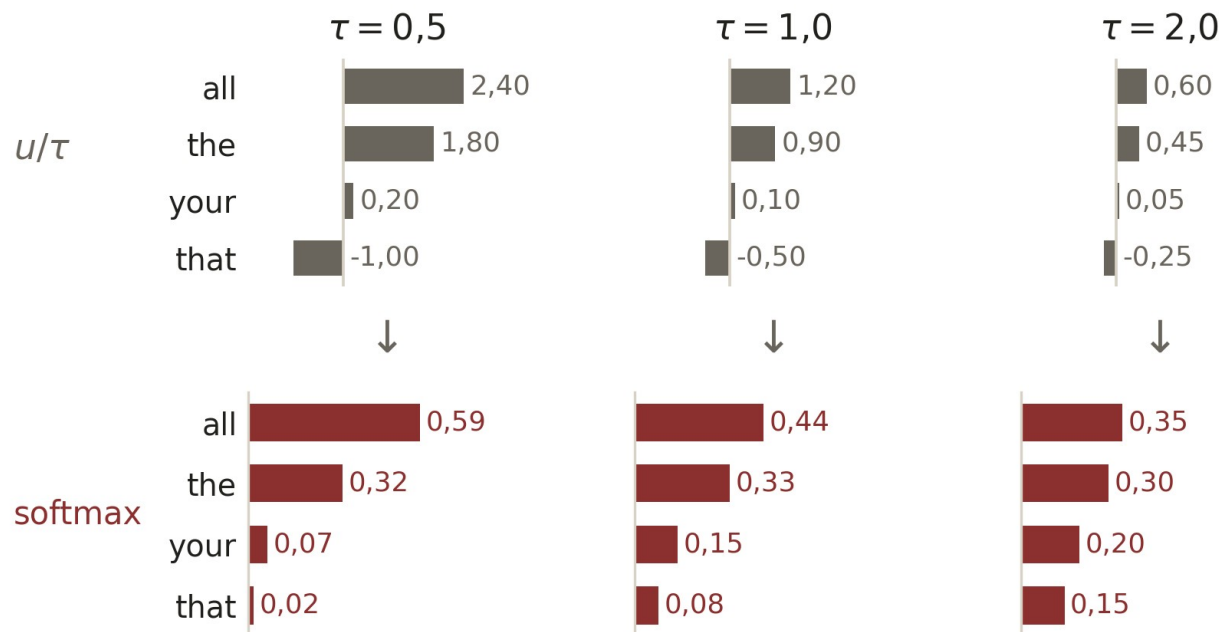
Mistral-7B, distribución del token siguiente para «The students opened their».

La softmax con temperatura

La distribución del token siguiente se calcula con los logits divididos por τ . Con $\tau < 1$ los logits se separan y la masa se concentra; con $\tau > 1$ se acercan y la masa se reparte.

$$P(w_{t+1} = w \mid w_{\leq t}) = \frac{e^{u_t[w]/\tau}}{\sum_{v \in \mathcal{V}} e^{u_t[v]/\tau}} \quad u_t: \text{ los logits de la posición } t$$

«So long and thanks for ...», el token siguiente con cuatro palabras en el vocabulario



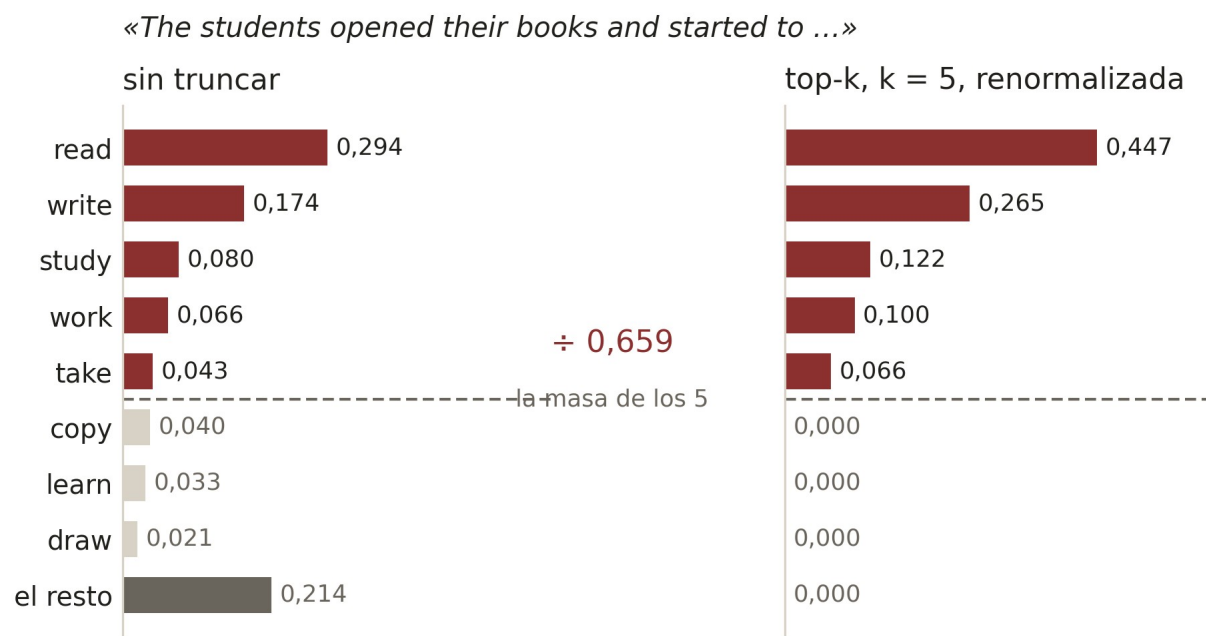
Top-k sampling

Se conservan los k tokens más probables y se renormalizan. Todos ganan probabilidad de muestreo, también los que no están en la cabeza.

$\mathcal{V}^{(k)}$ = las k piezas más probables tamaño fijo

$$P'(w) = \frac{P(w \mid w_{<t})}{\sum_{v \in \mathcal{V}^{(k)}} P(v \mid w_{<t})}$$

se renormaliza y se muestrea de P'



Top-p sampling

El mismo procedimiento, pero el corte se fija por masa de probabilidad y no por número de tokens. Se conservan los más probables hasta juntar p.

$$\mathcal{V}^{(p)} = \text{el conjunto más chico con } \sum_{w \in \mathcal{V}^{(p)}} P(w \mid w_{<t}) \geq p \quad \text{tamaño variable}$$
$$P'(w) = \frac{P(w \mid w_{<t})}{\sum_{v \in \mathcal{V}^{(p)}} P(v \mid w_{<t})} \quad \text{se renormaliza y se muestrea de } P'$$

